

GG-Core

Cahier des charges

1. Présentation générale.....	4
b. Contexte du projet.....	4
c. Objectifs généraux.....	4
Automatiser la gestion des matchs.....	4
Centraliser la supervision des serveurs CS2.....	4
Fournir une interface web unifiée.....	5
Garantir la fiabilité et la résilience du système.....	5
Préparer l'évolution vers un écosystème complet.....	5
d. Portée du projet (périmètre inclus/exclus).....	6
Périmètre inclus.....	6
Périmètre exclus.....	6
e. Parties prenantes et rôles.....	7
Partie prenante.....	7
Rôle / Responsabilité principale.....	7
2. Description du besoin.....	8
b. Problématique à résoudre.....	8
c. Public cible / utilisateurs finaux.....	9
d. Enjeux et critères de réussite.....	9
Enjeux.....	9
Critères de réussite.....	9
e. Contraintes (techniques, performance, LAN sans Internet, etc.).....	10
Contraintes techniques.....	10
3. Fonctionnalités attendues.....	11
b. Gestion des tournois et matchs.....	11
c. Gestion des équipes et joueurs.....	12
d. Contrôle des serveurs (start, pause, knife, swap, overtime...).....	13
e. Collecte et affichage des statistiques (kills, deaths, assists, %HS, economy, etc.).....	14
f. Système de pauses tactiques/techniques.....	16
Types de Pauses.....	16
Fonctionnement Technique.....	16
Critères de Réussite.....	17
g. Interface admin, joueur, publique.....	17
Interface Administrateur.....	17
Interface Joueur.....	18
Interface Publique.....	18
h. Export / historique des matchs.....	19
i. Évolutions possibles.....	20
j. Scénarios de validation.....	21
4. Exigences techniques.....	22
a. Architecture logicielle.....	22
b. Communication inter-services.....	22
c. Contraintes réseau.....	23
d. Sécurité et authentification.....	23
e. Persistance et sauvegarde.....	24

f. Déploiement et conteneurisation.....	24
g. Observabilité et diagnostic.....	24
5. Interfaces et ergonomie.....	25
a. Principes de conception.....	25
b. Interface administrateur.....	25
c. Interface publique.....	26
d. Interface joueur (prévue v1.1).....	26
e. Compatibilité et accessibilité.....	26
f. Critères de réussite.....	26
6. Données et Stockage.....	27
a. Schéma de base de données.....	27
c. Gestion du Temps Réel.....	27
d. Volumétrie Estimée.....	27
7. Exigences non fonctionnelles.....	29
a. Maintenabilité et extensibilité.....	29
c. Portabilité et déploiement.....	29
d. Observabilité.....	29
e. Conformité et données.....	29
g. Résilience aux erreurs.....	29
9. Lexique étendu.....	30
10. Annexes.....	32
a. Schéma d'architecture en partant du serveur.....	32
b. Schéma d'architecture en partant du front end.....	33
c. Hierarchie du projet (https://github.com/DocR3D/GG-Core) :.....	34
Couche Adapters (I/O).....	34
Couche Application (use cases et services).....	35
Couche Domaine (contrats purs).....	36

1. Présentation générale

b. Contexte du projet

Le projet **GG-Core** s'inscrit dans la continuité du célèbre outil **eBot** utilisé pour l'arbitrage et la gestion de matchs sur Counter-Strike.

Les outils actuels de gestion de matchs sur Counter-Strike 2, tels que l'historique eBot, restent fonctionnels mais vieillissant et ne couvrent pas tout le périmètre d'une Lan.

L'objectif est donc de **reconstruire un nouveau gestionnaire de tournois** moderne, automatisé et performant, adapté aux contraintes actuelles :

- Fonctionnement sans plugin serveur pour ne pas impacter les performances des serveurs CS2.
- Capacité à piloter et suivre plusieurs serveurs simultanément (jusqu'à 32).
- Collecter en temps réel des logs et événements du jeu (kills, rounds, economy, pauses, etc.).
- Interface web unifiée pour administrer et créer les matchs, suivre le score en direct et afficher les informations au public et aux joueurs.
- Compatibilité LAN (fonctionnement hors ligne ou avec réseau local uniquement).
- Compatibilité potentielle WAN

Le projet vise à répondre aux besoins d'organisation d'événements e-sport tels que les **GG-Lan à Brest**, où des matchs CS2 se déroulent simultanément, avec affichage des scores en direct, gestion fluide des pauses et redémarrages, et une centralisation des statistiques pour la communication post-événement.

c. Objectifs généraux

Automatiser la gestion des matchs

- Création, planification et suivi des matchs.
- Création et réalisation des véto
- Gestion automatique des phases de jeu : knife, swap, overtime, pauses tactiques et techniques.
- Synchronisation des scores et statistiques en temps réel.

Centraliser la supervision des serveurs CS2.

- Piloter plusieurs serveurs via RCON.
- Réception des logs HTTP pour collecter les événements du jeu.
- Assurer un suivi stable

Fournir une interface web unifiée

- **Interface administrateur** : gestion des équipes, joueurs, tournois, et contrôle des serveurs.
- **Interface publique** : affichage des scores, statistiques, et messages sponsorisés en direct.

Garantir la fiabilité et la résilience du système

- Sauvegarde des données en base relationnelle pour l'historique et les replays.
- Tolérance aux pannes : reprise automatique après crash d'un agent.

Préparer l'évolution vers un écosystème complet

- Extension possible vers la collecte de données CSTV (positions, grenades, heatmaps, replays 2D).
- Intégration future avec un système de diffusion d'overlay pour le stream.
- Possibilité d'ajouter des modules statistiques avancés (rating HLTV, clutch %, entry %, etc.).
- Export des statistiques sous forme d'image.

d. Portée du projet (périmètre inclus/exclus)

Périmètre inclus

Le projet **GG-Core** couvre l'ensemble des éléments nécessaires à la gestion complète d'un tournoi Counter-Strike 2 :

Gestion des entités de tournoi :

- Tournois, équipes, joueurs, vétos, matchs et maps.

Contrôle des serveurs de jeu :

- Communication via RCON (start, restart, pause, swap, overtime, etc.).
- Réception et parsing des logs HTTP envoyés par CS2.

Verrouillage des accès

- Chaque match est **protégé par un mot de passe unique obligatoire**, généré automatiquement lors de sa création.
- Une **whitelist SteamID** peut être activée en complément pour vérifier que seuls les joueurs autorisés rejoignent le serveur ou pour exclure certains profils.
- L'interface joueur propose un bouton « **Se connecter** » qui applique automatiquement le mot de passe du match.

Collecte et traitement des données de match :

- Kills, assists, deaths, score, économie, round states, pauses, phases de jeu.
- Agrégation des statistiques simples par joueur, équipe et tournoi.

Architecture temps réel et centralisée :

- Redis pour le dispatch d'événements.
- Base de données pour la persistance et l'historique.

Interfaces web :

- **Admin** : création et gestion des tournois, équipes, serveurs et matchs.
- **Public** : affichage live des scores, stats et messages sponsorisés.

Déploiement simplifié :

- Fonctionnement via Docker Compose, sur LAN ou Internet.
- Support multi-serveurs (jusqu'à 32/64 simultanés).

Périmètre exclus

Certaines fonctionnalités ne font pas partie du périmètre initial du projet :

fournir des overlays des streams

Replay 2D ou viewer interactif Live (prévu dans une phase ultérieure via CSTV).

Collecte et traitement des données compliqué de match :

- Eco Kill, nombre de dégâts par round, Entry kill

e. Parties prenantes et rôles

Partie prenante	Rôle / Responsabilité principale
Équipe technique GG-Core	Conception, développement et maintenance du système complet (backend, frontend, agents Go, infrastructure).
Administrateurs de tournoi (CS Admins)	Utilisateurs principaux de l'interface admin. Gèrent les matchs, les véto, les pauses, les swaps et les scores pendant l'événement.
Spectateurs / Public LAN	Consultent les résultats, scores et statistiques en direct via l'interface publique.
Joueurs	Interagissent indirectement via les serveurs CS2 (leurs actions génèrent les événements collectés par GG-Core). E
Administrateur système	Déploie et maintient l'infrastructure (serveurs Docker, Redis, PostgreSQL, agents Go, réseau LAN).

2. Description du besoin

Cette partie décrit la problématique à résoudre, les utilisateurs visés et les contraintes identifiées. Elle précise pourquoi le projet est nécessaire et sur quels critères son succès sera évalué.

b. Problématique à résoudre

Les outils actuels de gestion de matchs sur Counter-Strike 2, tels que l'historique eBot, restent fonctionnels via le protocole logaddress_add, mais présentent plusieurs limites :

- Architecture vieillissante (plus de dix ans) difficile à maintenir et à faire évoluer; technologies obsolètes, non adaptées à une architecture distribuée moderne;
- Écosystème fragmenté : plusieurs outils sont nécessaires pour gérer les matchs, les serveurs, les statistiques et les overlays ;
- Absence de fonctionnalités clés comme la création automatique d'arbre ou de poules, la gestion des vétos de maps;
- Impossibilité d'exploiter des sources de données avancées comme CSTV pour la capture des positions, grenades et replays.

Dans ce contexte, GG-Core vise à proposer une solution complète, unifiée et moderne, capable de gérer à la fois le pilotage des serveurs, la gestion des tournois, et la collecte de données temps réel, tout en préparant le système à des fonctionnalités avancées futures basées sur CSTV.

Les organisateurs doivent pouvoir gérer la totalité du cycle de tournoi, depuis les inscriptions jusqu'à la finale :

- Création automatique des poules et arbres ;
- Configuration et réalisation des vétos de maps ;
- Génération automatique des matchs à partir des résultats ;
- Affichage pour chaque joueur de sa page personnelle (prochain match, heure, serveur).

c. Public cible / utilisateurs finaux

Utilisateur	Besoin principal	Usage du système
Administrateurs de tournoi	Gérer les matchs, les vétos, les serveurs et les équipes depuis une interface unique	Utilisent l'interface admin pour créer, lancer, mettre en pause ou clôturer les matchs, suivre les scores et gérer les situations spéciales (swap, overtime, veto, etc.)
Joueurs	Participer aux matchs dans un environnement fluide et équitable	Interagissent indirectement via les serveurs CS2 ; leurs actions génèrent les données collectées par GG-Core
Spectateurs et public LAN	Suivre les scores et statistiques en direct	Accèdent à la page publique de suivi live et aux résumés post-match

d. Enjeux et critères de réussite

Enjeux

- **Modernisation de l'infrastructure** : Mettre à jour la gestion des tournois e-sport CS2 avec des technologies récentes, maintenables et extensibles.
- **Centralisation des opérations** : Regrouper toutes les fonctionnalités d'un tournoi (gestion des matchs, serveurs, statistiques, affichage) en un seul outil.
- **Optimisation des tournois LAN** : Réduire les temps d'attente et les erreurs humaines, et assurer une reprise rapide en cas d'incident.
- **Préparation à l'avenir** : Permettre l'intégration future de données CSTV (trajectoires, heatmaps, replays).
- **Amélioration de la visibilité** : Diffuser les scores et statistiques en direct pour le public.

Critères de réussite

- **Gestion des matchs** : Tous les matchs doivent être gérés et suivis via l'interface web, sans intervention manuelle sur les serveurs.
- **Stabilité du système** : Le système doit rester stable sur au moins 32 serveurs simultanés.
- **Fonctionnement hors ligne** : Le tournoi complet doit pouvoir être organisé en LAN sans connexion Internet.
- **Accessibilité des administrateurs** : Les administrateurs ne doivent pas avoir besoin de plugins SourceMod ni d'accès direct au serveur.
- **Fiabilité des données** : Le système doit enregistrer 100 % des rounds et kills, sans aucune perte de données.

e. Contraintes (techniques, performance, LAN sans Internet, etc.)

Contraintes techniques

- **Aucune dépendance serveur** : interdiction d'utiliser SourceMod ou tout plugin tiers pour ne pas altérer les performances CS2.
- **Compatibilité LAN** : le système doit fonctionner sans Internet, en environnement isolé, avec uniquement le réseau local.
- **Communication standardisée** : Les agents utilisent RCON et logaddress_add_http comme seules interfaces de communication avec CS2.
- **Faible latence** : les données doivent être traitées et affichées en temps réel.
- **Tolérance aux pannes** : un crash d'agent ou du backend ne doit pas nécessiter de redémarrage manuel des serveurs CS2. De la même manière, un crash d'agent doit être indépendant des autres agents.
- **Scalabilité locale** : gestion simultanée d'au moins 32 serveurs sur un même orchestrateur.
- **Architecture conteneurisée** : tout doit être exécutable sous Docker Compose pour faciliter le déploiement.

Contraintes organisationnelles

- Déploiement rapide et reproductible avant chaque LAN (setup complet < 15 min).
- Interfaces accessibles et compréhensibles par les administrateurs non techniques.
- Compatibilité avec les configurations existantes du site d'inscription des équipes.

Contraintes d'évolution

- Prévoir une intégration CSTV future (position, grenades, replays).
- Prévoir la possibilité de proposer dans le futur un overlay pour afficher sur les streams
- Maintenir un socle technologique moderne : Go, NestJS, Redis, PostgreSQL, Next.js.
- Respect des normes de sécurité réseau (auth, JWT, isolation des services).

3. Fonctionnalités attendues

Cette section présente les principales fonctions que doit assurer GG-Core pour garantir la gestion complète d'un tournoi CS2, depuis la création des matchs jusqu'à la visualisation publique des scores et statistiques.

b. Gestion des tournois et matchs

GG-Core doit permettre une **gestion complète et centralisée** des tournois de Counter-Strike 2, depuis la configuration initiale jusqu'à la fin du dernier match.

Création de tournoi :

- Nom, description, type (simple élimination, double élimination, poules, etc.).
- Configuration générale (format MR12, overtime, maps disponibles, nombre de serveurs).

Gestion des poules et arbres :

- Création automatique ou manuelle des groupes à partir de l'api Facelt si disponible.
- Génération d'un arbre à partir des résultats des poules.
- Mise à jour automatique des brackets en fonction des résultats.

Création et affectation des matchs :

- Réalisation des véto en ligne depuis l'interface admin, intégrée au déroulement du match avant le démarrage serveur.
- Lien entre équipes, maps et serveur CS2 assigné.
- Suivi de l'état du match : *à venir* / *en cours* / *terminé* / *en pause*.
- Historique des actions (swap, restart, pause, etc.).

Verrouillage des accès :

- Accès protégé par **whitelist SteamID** pour assurer que seuls les joueurs inscrits puissent rejoindre le serveur.
- Mot de passe unique généré à la création du match (utile si SteamID non disponible).
- L'interface joueur propose un bouton « Se connecter » qui applique automatiquement la méthode d'accès (SteamID et/ou mot de passe) selon la configuration du match.

Paramétrage des règles :

- Choix du nombre de rounds, du format MR12, MR15 ou MR3 (Overtime).
- Activation des prolongations et configuration du système d'économie.
- Paramètres spécifiques LAN (timeouts, messages sponsorisés, etc.)

Source de vérité et reconstitution d'état :

- Toutes les actions et événements sont enregistrés dans une timeline d'événements normalisée (Redis + PostgreSQL).
- Cette timeline permet de reconstruire l'état complet d'un match à n'importe quel instant (snapshot) et de pouvoir retirer des actions qui ne seraient pas vraiment arrivées.

Objectif : pouvoir sauvegarder, restaurer ou rejouer un match sans perte de cohérence.

Suivi en temps réel :

- Mise à jour du score et des rounds via WebSocket.
- Affichage des événements clés (round start/end, kills, bombes, pauses).
- Synchronisation automatique avec le front public et les overlays.

Archivage et historique :

- Sauvegarde des résultats et statistiques de chaque match.
- Export JSON/CSV ou affichage rétrospectif des données.

c. Gestion des équipes et joueurs

GG-Core est conçu pour être la colonne vertébrale des tournois, garantissant une gestion fluide et cohérente des équipes et des joueurs. Il assure une intégration parfaite entre les données d'inscription, les serveurs CS2 et la collecte de statistiques.

Gestion des Équipes :

- Créez, modifiez et supprimez des équipes.
- Associez des informations détaillées : nom, tag, logo, photo.
- Affectation intelligente aux poules et arbres de tournoi, manuelle ou automatique.

Gestion des Joueurs :

- Créez et éditez des profils joueurs complets (nom, pseudo, SteamID64, FaceltID).
- Attribuez les joueurs aux équipes.
- Liaison automatique et instantanée entre SteamID et les statistiques de jeu.

Synchronisation Serveur Avancée :

Liaison d'identité : les SteamID sont utilisés pour valider l'appartenance d'un joueur à un match (whitelist) et pour agréger les statistiques. Si SteamID indisponible, l'authentification par mot de passe est utilisée comme secours.

Statistiques Individuelles Approfondies :

- Suivi détaillé des performances : kills, deaths, assists, HS%, K/D, ADR, clutches, etc.
- Agrégation automatique et intuitive des données par match, par carte et par tournoi.

Expérience Utilisateur Optimale :

- Affichage des équipes et rosters sur l'interface publique.
- Présentation claire et simplifiée des comparaisons d'équipes (rating, etc.).

Données Joueurs Unifiées et Centralisées :

- Chaque joueur bénéficie d'un identifiant interne unique.
- Toutes les données (matches, statistiques, line-ups, événements) sont reliées à cet identifiant.
- Permet une historisation globale et une analyse transversale sur l'ensemble des tournois.

d. Contrôle des serveurs (start, pause, knife, swap, overtime...)

Pilotage et orchestration

- **Affectation serveur** : sélection auto/manuelle, contrôle d'un ou plusieurs serveurs en parallèle.
- **Démarrage match** : warmup, chargement map, config tournoi, vérifications préalables (slots, GOTV, convars clés).
- **Queue de commandes RCON** : ordonnancement, retry, timeout, idempotence, rate-limit.
- **Journal d'actions** : chaque commande est horodatée, corrélée au match et au serveur.

Phases de jeu

- **Knife round** : lancement, détection du vainqueur (simple ou détaillé), choix de side, application du swap.
- **Swap de sides** : mapping logique↔physique robuste, messages auto, vérifications score.
- **Overtime** : activation conditionnelle, économie et paramètres dédiés, cycles successifs.

Pauses et reprises

- **Pause tactique** : banque par équipe, compteur de secondes restantes, affichage live, auto-unpause à l'expiration.
- **Pause technique** : sans banque, envoie une notification aux admins.
- **Règles** : armement de pause à la prochaine phase d'achat, messages périodiques "time left", anti-abus.

Verrouillage des accès

- **Roster enforcement** : whitelist par mot de passe ou par steamID avec la possibilité aux joueurs de se connecter via un bouton sur leur interface.

Gestion de map et config

- **Changement de map** : say + countdown, sauvegarde d'état.
- **Templates de config** : LAN, knife, warmup, BO1/BO3, MR12/MR15, overtime on/off, overtime, presets sponsors/messages.

Sécurité et robustesse

- **Healthchecks** : heartbeat agent ↔ serveur, détection crash ou freeze.
- **Reprise** : re-attach après redémarrage agent/backend, resynchronisation snapshot. Mise en file dans l'agent les logs si le backend crash pour tout rattraper au redémarrage

Observabilité

- **Logs structurés** : commandes envoyées, réponses RCON, événements clés.
- **Événements temps réel** : match:state, round:start/end, pause:update, sides:swapped, score:update.

Critères d'acceptation

- Pause tactique avec **auto-unpause** fiable et message "N secondes restantes".
- Overtime activé automatiquement à égalité selon le format défini.
- Toute commande RCON confirmée ou réessayée, jamais exécutée deux fois.
- Reconnexion agent/backend sans perte de contrôle ni d'événements.

e. Collecte et affichage des statistiques (kills, deaths, assists, %HS, economy, etc.)

Collecte de données :

- **Source Principale** : Logs HTTP envoyés par `logaddress_add_http`.
- **Événements Suivis** :
 - Kills (arme, tir à la tête, équipe)
 - Assists, deaths (potentiellement dégâts)
 - Pose/désamorçage de bombe
 - Début/fin de round
 - Pausés, changements d'équipes, prolongations
 - Commandes de chat (`!pause`, `!ready`, etc.)
- **Validation et Parsing** : Normalisation via expressions régulières, puis publication dans Redis (clé : `ggcore:events:primary`).
- **Alignement Temporel** : Horodatage précis (tick + timestamp) pour assurer la cohérence entre les serveurs.

Traitement en Temps Réel :

- **Redis** sert de bus central, publiant chaque événement vers les consommateurs (backend, *stats worker*).
- **Backend NestJS :**
 - Enregistre les événements primaires (kills, rounds, économie).
 - Met à jour les projections (scoreboard joueur/équipe, round courant, économie).
- **Mise à Jour de l'UI :** Via WebSocket (événements `score:update`, `kill`, `round:end`).

Statistiques Calculées :

- **Individuelles :** Kills, deaths, assists, HS%, K/D, ADR, rating HLTV 1.0+/2.0 (selon les données disponibles), *entry kills*, *clutches*.
- **Par Équipe :** Rounds CT/T, *winrate*, succès éco, poses de bombe, désamorsages, bonus économiques.
- **Tournoi :** Classement général, meilleurs joueurs par catégorie, meilleure équipe par catégorie, vainqueur.

Affichage :

- **Interface Publique :**
 - Scoreboard en direct par match.
 - Graphiques simples (round-par-round, top kills).
 - Fiche joueur consultable avec statistiques cumulées.
- **Interface Joueur :**
 - Affichage des prochains matchs
 - Affichage des détails des matchs à venir
 - Notification quand un match est prêt avec un bouton pour se connecter
- **Interface Admin :**
 - Vue synthétique des scores + détails des rounds.
 - Historiques des événements.

Fiabilité et Reprise :

- Tous les événements sont horodatés et persistés (source de vérité).
- En cas de crash, le snapshot Redis + l'historique Postgres permettent de **rejouer les événements** pour reconstituer un match complet.
- Les pertes de connexion temporaires (agent ↔ backend) n'entraînent aucune suppression de données.

f. Système de pauses tactiques/techniques

GG-Core intégrera un système de pauses automatisé, fiable et transparent, conforme aux règles e-sport tout en étant adapté aux environnements LAN.

Types de Pauses

- **Pause Tactique**
 - Déclenchement : Par une équipe via commande en jeu (ex: `!pause`, `!tac`)
 - Durée : Limitée (ex: 5 minutes par match) avec un compteur visible.
 - Fin : Annulation automatique (auto-unpause) à la fin du compte à rebours ou via commande en jeu (ex: `!unpause`) ou intervention admin.
- **Pause Technique**
 - Déclenchement : Par une équipe via commande en jeu (ex: `!tech`, `!admin`) (pour problèmes matériels, réseau, etc.) ou depuis l'interface administrateur.
 - Durée : Illimitée jusqu'à reprise manuelle par un administrateur ou via commande en jeu (ex: `!unpause`).
 - Affichage : État visible sur l'interface publique et enregistré dans les logs.
 - Notification envoyé sur Discord ou sur le site

Fonctionnement Technique

- **Système d'Armement**
 - Une pause demandée est "armée" et se déclenche au début de la prochaine "freeze time" pour éviter d'interrompre les rounds actifs.
- **Banque de Pauses**
 - Chaque équipe dispose d'un compteur individuel de pauses tactiques restantes.
 - Stockage : Redis pour l'opérationnel, PostgreSQL pour la persistance et la cohérence post-redémarrage.
- **Gestion du Temps**
 - Le minuteur est géré côté backend, indépendamment du serveur de jeu.
 - Communication : Des événements `pause:update` sont émis chaque seconde via WebSocket pour synchroniser le frontend et les overlays.
 - Fin : L'auto-unpause se déclenche à l'expiration du minuteur.
- **Affichage et Communication**
 - Message automatique dans le chat du serveur de jeu (ex: "Pause tactique - Team A (30 sec restantes)").
 - Compteur visible sur les interfaces admin et publiques.
 - Notification sonore optionnelle pour les administrateurs.
- **Résilience**
 - En cas de redémarrage d'un agent ou du backend pendant une pause, le système reprend le minuteur restant.
 - Un snapshot Redis assure la continuité de l'état des pauses.

Critères de Réussite

- Aucune pause ne doit se déclencher pendant un round actif.
- Le minuteur tactique doit être précis à ± 5 secondes.
- L'auto-unpause doit fonctionner sans intervention manuelle.
- Les administrateurs doivent pouvoir reprendre un match à tout moment sans désynchronisation.
- L'état de la pause doit rester cohérent entre le serveur de jeu, le backend et le frontend.

g. Interface admin, joueur, publique

Interface Administrateur

L'outil principal pour les arbitres et organisateurs de LAN, offrant un contrôle complet du tournoi sans interaction directe avec les serveurs CS2.

Fonctionnalités Clés :

Tableau de Bord Administrateur

- Créer un nouveau tournois
- Modifier un tournois (Forcer le passage en arbre, changer le format)

Tableau de Bord du Tournoi

- Vue d'ensemble des tournois, matchs, équipes et serveurs actifs.
- Statut en temps réel : *en cours* / *terminé* / *en attente* / *en pause* / *En retard*.

Console de Match

- Commandes directes (démarrer, redémarrer, pause, reprendre, échanger, changer de carte, restaurer).
- Flux d'événements en temps réel (rounds, éliminations, messages de chat).
- Contrôle manuel du tableau des scores pour d'éventuelles corrections.

Gestion des Entités

- Création et édition des équipes, joueurs, matchs et serveurs.
- Attribution automatique ou manuelle des serveurs aux matchs.
- Visualisation de la composition et verrouillage par SteamID.

Gestion des Sponsors et Affichage Public

- Ajout et modification de messages sponsorisés à afficher dans le chat ingame.
- Rotation manuelle ou automatique des messages.

Journaux et Statistiques

- Historique détaillé de chaque match (chronologie, statistiques agrégées).

Exigences Techniques

- Authentification JWT avec gestion des rôles (administrateur / personnel / spectateur).
- Interface réactive, optimisée pour tablette et ordinateur portable.
- Reconnexion automatique aux WebSockets en cas de perte réseau.

Interface Joueur

Offrir aux joueurs un espace minimaliste pour s'identifier, consulter leur emploi du temps, recevoir des notifications en cas de match prêt, avoir un bouton pour faciliter la connexion et afficher leur statistiques.

Identification

- Liaison SteamID (lecture seule)

Centre de match

- Affichage de l'heure, du serveur et de la carte
- Notifications en temps réel des changements d'horaire.
- Notification quand le match est prêt à être rejoint avec un bouton pour lancer la connexion

Vérifications de cohérence (Sanity checks)

- **Vérification d'accès** : l'interface affiche une alerte si le SteamID du joueur n'est pas présent dans la whitelist du match.

Exigences Techniques

- Rôles "joueur" distincts des administrateurs
- Authentification LAN légère, sans dépendance externe (avoir une alternative au steamID en cas de perte de connexion internet)
- Lecture des données en temps réel via WebSocket, sans commandes serveur

Interface Publique

Conçue pour les spectateurs sur place et à distance, ou pour les joueurs souhaitant voir des informations concernant les matchs des autres équipes. Il doit offrir un affichage clair, réactif et sans latence perceptible.

Page "Match en Direct"

- Tableau des scores complet : équipes, scores, rounds, phase actuelle.
- Statistiques détaillées des joueurs (K/D/A, % Tête, ADR).
- Chronomètre de round et état de la bombe (posée, désamorcée, explosée).
- Messages sponsorisés dynamiques dans le chat in-game.

Page "Tournoi"

- Liste des matchs en cours et terminés.
- Accès direct à la page live de chaque match.
- Classements et statistiques globales.

Page "Équipe / Joueur"

- Profil avec historique des matchs et statistiques cumulées.

Exigences Techniques

- Synchronisation WebSocket en temps réel.
- Mode plein écran sans rechargement de page.
- Design réactif et lisible sur grands écrans.

Critères de Réussite

- L'interface administrateur doit être capable de piloter un match complet sans recours à la console serveur.
- La navigation doit être fluide et sans rafraîchissement.
- L'authentification et les rôles doivent être fonctionnels, empêchant tout accès public aux commandes critiques.

h. Export / historique des matchs

Le module GG-Core est conçu pour assurer la persistance et la consultation des données de matchs, rounds et événements.

Archivage Automatique et Fidélité des Données

À l'issue de chaque match, le backend procède à une sauvegarde exhaustive des informations suivantes :

- Scores finaux
- Statistiques détaillées des joueurs et équipes
- Chronologie complète des événements (éliminations, pauses, remplacements, etc.)
- Économie par round
- Configuration précise du match (carte, équipes, serveurs, version du jeu)

Toutes ces données sont stockées de manière fiable dans **PostgreSQL**, constituant ainsi une vérité historique inaltérable et exportable via l'interface d'administration. La capacité de rejouer ou de reconstruire un match à partir de sa chronologie est une fonctionnalité clé, garantissant l'intégrité de la vérité historique.

Consultation Intuitive des Données

Interface Administrateur :

- Liste complète des matchs passés, enrichie de filtres avancés (tournoi, équipe, carte, date) pour une recherche aisée.
- Vue détaillée de chaque match : tableau des scores finaux, statistiques approfondies et historique round par round.

Interface Publique :

- Résumés synthétiques des matchs : score, meilleur joueur, date et carte jouée, offrant une vue rapide et pertinente.
- Visualisation des statistiques globales d'un tournoi, permettant une analyse macroscopique des performances.

Critères de Réussite Essentiels

- Garantie d'aucune perte de données de match, même après un redémarrage du backend.
- L'historique complet d'un tournoi doit rester intégralement consultable et rejouable.

i. Évolutions possibles

L'objectif de ce sous-chapitre est de lister les fonctionnalités ou extensions envisagées à moyen ou long terme. Bien que non incluses dans la première version, ces anticipations visent à orienter la conception technique du projet.

CSTV (Counter-Strike TV+) :

- Collecte des positions, trajectoires de grenades et replays.
- Génération de replays 2D interactifs sur mini-carte.

Export :

- Formats JSON/CSV pour publication post-LAN ou archivage.
- Création automatique de fiches joueurs et équipes au format image (pour publication sur les réseaux sociaux).
- Export infographique de tableaux de scores ou de statistiques.

Overlays stream avancés :

- API WebSocket pour une intégration directe dans OBS/XSplit.
- Gestion dynamique des sponsors, transitions et animations.

Statistiques enrichies :

- Calculs de Rating HLTV 3.0, clutch%, entry%, trade ratio.
- Analyse comparative entre joueurs ou équipes.

Module de planning et gestion complète des tournois :

- Gestion automatique des horaires, retards et notifications administrateur.

Système de replay global :

- Archivage automatique des démos GOTV.
- Visionneuse interne ou export vers un visualiseur externe.

j. Scénarios de validation

- Lancer un tournoi de 10 serveurs simultanés sans perte d'événements.
- Redémarrer le backend : reprise des matchs en cours depuis snapshots.
- Tester une pause tactique et vérifier auto-unpause à la fin de la banque.
- Simuler crash agent : reconnexion sans perte de contrôle.
- Simuler crash server CS : création de fichier backup à partir de la snapshot

4. Exigences techniques

Ce chapitre détaille les fondations techniques du projet, incluant les composants principaux, la communication inter-services, et les exigences non fonctionnelles (performance, sécurité, résilience).

a. Architecture logicielle

Architecture distribuée composée de cinq blocs :

- **Agents Go** : Les agents exécutent les commandes RCON reçues via Redis et publient les logs HTTP de CS2.
- **Backend NestJS** : Logique métier, API REST, WebSocket, synchronisation.
- **Redis** : Bus de messages temps réel (Pub/Sub) et cache.
- **PostgreSQL** : Persistance longue durée (vérité historique).
- **Frontend Next.js** : Interface d'administration et interface publique.

Flux de données

- Frontend (Admin) → Backend NestJS → Redis → Agent Go → Serveur CS2 → Agent Go → Redis → Backend → Frontend (Public/Admin). (Voir annexe)
- Sauvegarde parallèle des événements vers PostgreSQL.

Philosophie

- L'événement est l'unité centrale.
- Les snapshots sont reconstruits à partir des événements (pattern event sourcing).
- Chaque module est indépendant et tolérant aux pannes.

b. Communication inter-services

RCON : Commandes vers les serveurs CS2 (start, pause, restart, etc.).

HTTP Logs : Les agents reçoivent des logs via `logaddress_add_http` et les renvoient sous formes d'événement sur Redis.

Redis (Pub/Sub) : Distribution des événements normalisés.

HTTP REST API (NestJS) :

- Gestion du tournoi, des équipes et des serveurs.
- Endpoints publics : `/api/matches/:id`, `/api/teams`, etc.

WebSocket (Socket.IO) : Diffusion temps réel vers le frontend.

c. Contraintes réseau

Fonctionnement LAN-first : Aucun service ne dépend d'Internet.

Compatibilité IPv4/IPv6.

Ports standards :

- RCON : 27015/tcp
- GOTV : 27020/udp (optionnel)
- Backend HTTP : 8081
- Redis : 6379
- Postgres : 5432

d. Sécurité et authentification

Contrôle d'accès principal : chaque match possède un mot de passe unique obligatoire, régénéré à chaque création.

Contrôle d'accès complémentaire : une whitelist SteamID optionnelle peut être utilisée pour restreindre ou bloquer certains joueurs.

Interface joueur : le bouton « Se connecter » applique automatiquement le mot de passe du match.

Sécurité générale : rotation des secrets JWT et RCON, whitelists IP sur [/cs2/logs](#), et non-exposition de Redis/Postgres en dehors du LAN.

Authentification JWT avec rôles (admin / joueur/ publique).

Séparation réseau : Redis et PostgreSQL non exposés publiquement.

Chiffrement HTTPS pour toutes les interfaces web externes.

Validation stricte des payloads côté API.

Logs d'accès et Journalisation des actions critiques (pause, restart, unpause).

Durcissement en entrée : validation stricte schémas, rate-limit RCON/HTTP.

Whitelist IP pour envoyer des logs sur [/cs2/logs](#).

Secret RCON unique par serveur, régénéré avant chaque LAN.

Rotation mensuelle des JWT secrets.

Limitation du nombre de requêtes RCON par seconde.

e. Persistance et sauvegarde

Redis : Cache et file d'événements actifs (durée de vie limitée).

PostgreSQL : Vérité historique persistante (événements, entités, snapshots).

Sauvegardes automatiques : Dump quotidien de la base et persistance des volumes Docker.

Reconstruction d'état : Possibilité de restaurer un match à partir des événements stockés.

f. Déploiement et conteneurisation

Docker Compose : Exécution des services (agent, backend, frontend, redis, postgres).

Volumes persistants pour Redis et PostgreSQL.

Configuration réseau : Bridge Docker isolé, sous-réseaux distincts par environnement (LAN / prod).

Logs centralisés : Sortie standard + fichiers de rotation (format JSON).

g. Observabilité et diagnostic

Metrics internes : Latence, TPS, nombre d'événements traités, erreurs RCON.

Healthchecks sur tous les services (`/health` endpoint).

Alerting simple (Redis TTL, heartbeats agents).

Logs structurés : Horodatage, type, `matchId`, `serveurId`, niveau (info/warn/error).

5. Interfaces et ergonomie

GG-Core est conçu pour offrir des interfaces utilisateur claires, réactives et cohérentes. Optimisées pour une utilisation en réseau local (LAN).

a. Principes de conception

Lisibilité immédiate : Les informations cruciales (score, round, timer, état de pause) sont mises en avant pour une compréhension instantanée.

Action rapide : Les commandes essentielles sont directement accessibles, éliminant le besoin de navigations complexes.

Retour visuel systématique : Chaque interaction (pause, échange d'équipes, redémarrage) génère un feedback visuel instantané et explicite.

Uniformité visuelle : Une charte graphique cohérente assure une expérience unifiée sur les interfaces administrateur, publique et joueur.

Mode clair/sombre adaptatif : L'interface s'adapte à l'environnement d'utilisation (salle LAN, scène d'événement, bureau).

Ergonomie "LAN" : Des boutons larges, des raccourcis clavier intuitifs et une compatibilité tactile (tablette) garantissent une manipulation aisée.

b. Interface administrateur

Tableau de bord global : Une vue synthétique de l'ensemble des tournois, des matchs, des serveurs et de leurs statuts.

Console de match : Un espace de contrôle dédié affichant les logs en temps réel, le tableau des scores et des boutons d'action instantanés (pause, échange d'équipes, redémarrage).

Vue tournoi : Une arborescence claire des poules et des phases finales avec une progression automatique.

Vue statistiques : Des graphiques détaillés par équipe, joueur et carte pour une analyse approfondie.

Exigences UX :

- **Navigation fluide** : Pas de rechargement de page grâce à l'utilisation de Next.js et de WebSockets.
- **État visuel clair** : Un code couleur intuitif pour indiquer les statuts (vert = actif, orange = en pause, rouge = erreur).
- **Reconnexion automatique** : Gestion robuste des pertes de connexion réseau avec reconnexion automatique aux sockets.

c. Interface publique

Page Match : Un tableau de score dynamique, un timer précis, l'affichage de la phase de jeu actuelle et des messages sponsorisés.

Page Tournoi : Une liste complète des matchs terminés et en cours, avec des liens directs vers les pages live correspondantes.

Page Équipe/Joueur : Des statistiques globales et les performances récentes de chaque équipe et joueur.

d. Interface joueur

Page personnelle : Intégration du lien Steam et du lien faceIT, planning des matchs

Notification de match : Réception de notification lorsque le serveur est prêt avec un bouton pour le rejoindre

Lecture seule : Accès aux informations de match sans aucune commande critique.

e. Compatibilité et accessibilité

Navigateurs pris en charge : Compatibilité étendue avec Chrome, Edge et Firefox.

Responsive design : Une interface optimisée pour PC, tablette et TV.

f. Critères de réussite

Compréhension intuitive : Scoreboard et commandes administrateur utilisables sans formation préalable.

Synchronisation parfaite : Cohérence visuelle avérée entre les interfaces administrateur, publique et joueur.

Exploitabilité optimale : L'interface doit être pleinement fonctionnelle sur scène LAN sans aucune perte de lisibilité.

6. Données et Stockage

GG-Core repose sur une architecture événementielle et relationnelle. Les données brutes issues des logs CS2 sont transformées en événements structurés, stockés dans PostgreSQL pour l'historique et diffusés via Redis pour le temps réel.

a. Schéma de base de données

Tables principales :

- **tournaments** : métadonnées du tournoi (nom, format, date, règles).
- **teams** : équipes, logo, tag.
- **players** : identifiants internes (UUID), SteamID, pseudo, faceltID, équipe associée.
- **matches** : entité centrale (équipes, map, score, état).
- **events_primary (Vérité absolue)** : événements primordiaux (kills, début/fin de round, bombe, chat).
- **events_secondary** : télémétrie (positions, grenades, CSTV).
- **scoreboard_player_match / team_match** : projections temps réel.

Relations :

- `players.team_id` → `teams.id`
- `matches.tournament_id` → `tournaments.id`
- `events_primary.match_id` → `matches.id`
- `scoreboard_*` matérialisées à partir des événements.

Partitionnement :

Les tables `events_*` sont partitionnées par `tournament_id` pour faciliter la purge et les requêtes.

c. Gestion du Temps Réel

Redis Pub/Sub : diffusion immédiate des événements entre services.

Redis Streams (optionnel) : persistance courte durée pour tolérance aux pannes.

Snapshot : Structure sérialisée du match courant (scores, côtés, pauses, rounds) servant à restaurer l'état d'un match après redémarrage.

d. Volumétrie Estimée

Élément	Taille par match	Taille par tournoi (10 matchs)
Événements primaires	~50 000 lignes	~500 000 lignes
Scoreboard joueurs	~20 ko	~200 ko
Snapshots	~100 ko	~1 Mo
Total (par tournoi)		≈ 300–400 Mo

7. Exigences non fonctionnelles

a. Maintenabilité et extensibilité

Code modulaire (agents Go, backend NestJS, front Next.js).

Contrats stables (DTO/Events versionnés).

Projections régénérables à partir des événements.

c. Portabilité et déploiement

Docker Compose pour LAN/prod.

Zéro dépendance Internet en mode LAN.

d. Observabilité

Logs structurés JSON avec corrélation matchId, serverId, seq.

Healthchecks /health pour chaque service.

Métriques : latence event→WS, backlog Redis, RPS, erreurs RCON.

e. Conformité et données

RGPD minimal : aucune donnée sensible ; pseudonymes joueurs + SteamID.

Politique de rétention configurable (purge tournois anciens).

Exports traçables (horodatage, hash optionnel).

Suppression ou export possible sur demande d'un joueur.

g. Résilience aux erreurs

Idempotence des commandes RCON.

Mode dégradé : lecture sur snapshots si projections manquantes.

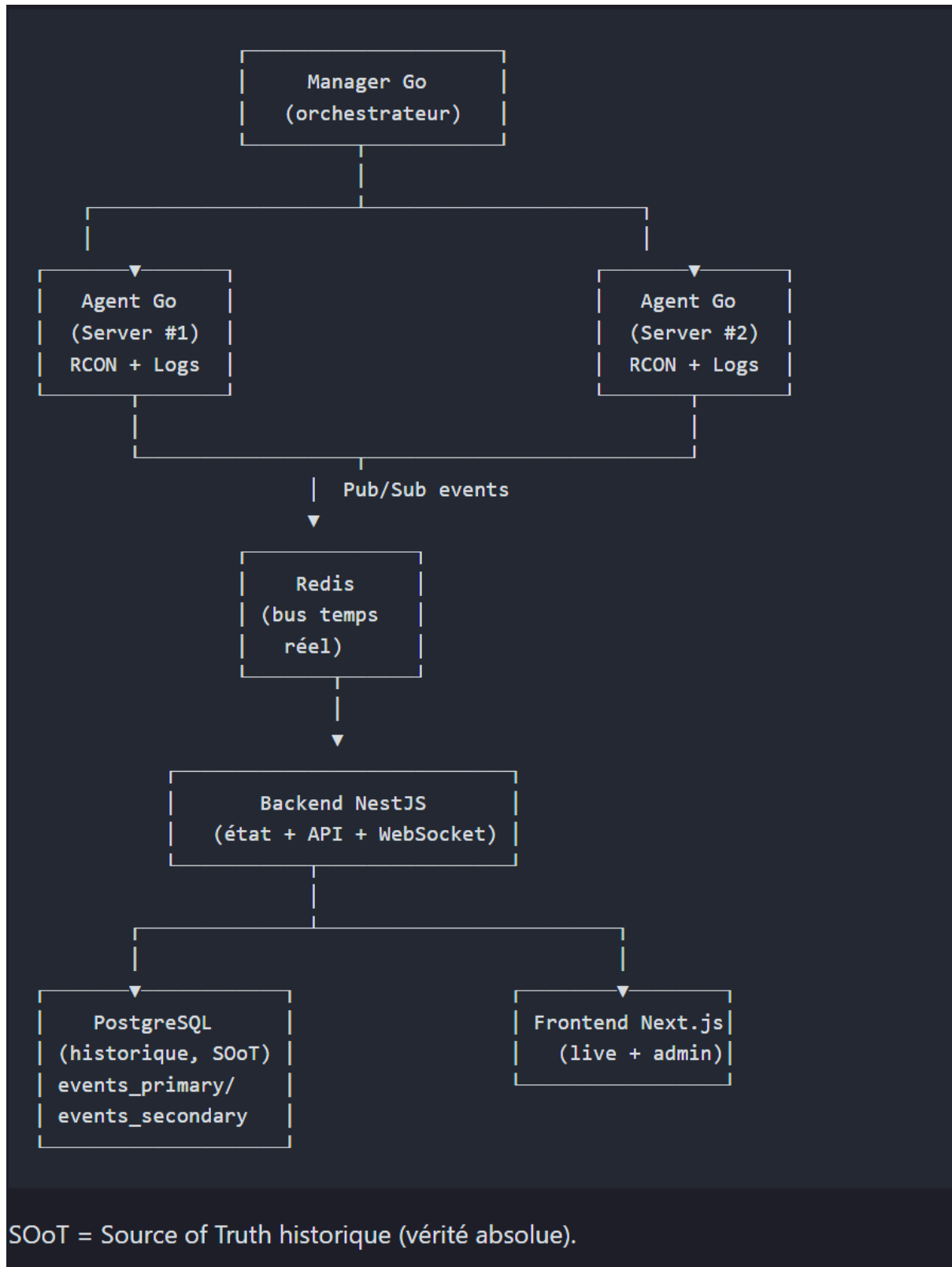
9. Lexique étendu

- **ADR (Average Damage per Round)** : Dégâts moyens infligés par round, indicateur de performance.
- **Agent** : Programme (Go) connecté à un serveur CS2, exécutant les commandes RCON, collectant les logs HTTP et les publiant dans Redis
- **Armor / Kevlar / Helmet** : Équipement de protection réduisant les dégâts reçus.
- **Backend (NestJS)** : Serveur d'application gérant logique, API et WebSockets.
- **Bomb plant / defuse** : Pose ou désamorçage de la bombe, objectif central du jeu.
- **Bracket** : Arbre de tournoi (simple ou double élimination) montrant la progression des équipes.
- **Buy / Économie** : Phase d'achat d'armes et équipements avant un round.
- **Clutch** : Situation où un joueur seul gagne un round contre plusieurs adversaires (ex : 1v3).
- **CT (Counter-Terrorist)** : Camp défensif, empêche la pose ou désamorce la bombe.
- **CSTV / GOTV** : Système d'observation et d'enregistrement de matchs CS2, transmet positions et grenades.
- **Demo** : Fichier d'enregistrement complet d'un match (replay).
- **Drop** : Action de donner une arme à un coéquipier.
- **Eco / Eco round** : Round avec peu d'achat pour économiser.
- **Eco-kill** : Élimination avec arme faible durant un round éco.
- **Entry kill** : Première élimination d'un round.
- **Entry trade** : Réponse immédiate après la mort d'un coéquipier.
- **Flash** : Grenade aveuglante désorientant les adversaires.
- **Force buy** : Achat forcé malgré faible économie.
- **Freeze time** : Temps d'attente avant le début d'un round.
- **HS (Headshot)** : Tir à la tête, inflige plus de dégâts.
- **HUD (Heads-Up Display)** : Interface du jeu affichant score, timer, etc.
- **Knife round** : Round au couteau servant à déterminer les sides de départ.
- **K/D (Kill/Death ratio)** : Ratio kills/morts d'un joueur.
- **LAN** : Tournoi en réseau local sans Internet.
- **Map** : Carte de jeu (Mirage, Inferno, Nuke...).
- **Match / Round** : Ensemble de rounds formant un match.
- **Molotov / Incendiary** : Grenade incendiaire bloquant une zone.
- **Money** : Argent accumulé par joueur pour acheter son équipement.
- **Overtime** : Prolongation en cas d'égalité (ex : 12–12).
- **Pause tactique** : Pause courte demandée par une équipe pour discuter stratégie.
- **Pause technique** : Pause longue décidée par un admin pour problème matériel ou réseau.
- **Pick / Veto** : Sélection ou élimination de cartes avant le match.
- **Pistol round** : Premier round joué uniquement avec armes de poing.
- **Plant** : Pose de la bombe par les Terrorists.
- **RCON (Remote Console)** : Protocole pour contrôler à distance les serveurs CS2.
- **Redis** : Base en mémoire servant de bus temps réel.
- **Retake** : Reprise de site après la pose de la bombe.
- **Round bonus / loss bonus** : Compensation économique pour équipe perdante.
- **Scoreboard** : Tableau des statistiques (kills, deaths, assists...).

- **Smoke** : Grenade fumigène bloquant la vision.
- **Snapshot** : Image instantanée de l'état du match (score, pauses, économie).
- **Swap** : Échange des côtés CT/T à la mi-temps ou après un knife.
- **T (Terrorist)** : Camp offensif chargé de poser la bombe.
- **Timeout** : Synonyme de pause tactique.
- **Trade** : Élimination d'un adversaire juste après la mort d'un coéquipier.
- **Triple / Quad / Ace** : 3, 4 ou 5 kills dans un round par un seul joueur.
- **Utility** : Ensemble des grenades et équipements.
- **WebSocket** : Canal bidirectionnel temps réel entre backend et frontend.
- **Wallbang** : Tir à travers un mur ou un obstacle mince.
- **Whitelist / SteamID** : Mécanisme complémentaire de sécurité listant les SteamID autorisés à rejoindre un match. Le mot de passe du match est toujours obligatoire, la whitelist SteamID peut s'y ajouter pour restreindre davantage les accès ou bloquer des joueurs précis.

10. Annexes

a. Schéma d'architecture en partant du serveur



b. Schéma d'architecture en partant du front end

